

Remote Monitoring System of Patient Status in Social IoT Environments Using Amazon Web Services Technologies and Smart Health Care

Amer Tahseen Abu-Jassar¹, Hani Attar^{2,3} ✉, Ayman Amer², Vyacheslav Lyashenko⁴, Vladyslave Yevsieiev⁴, and Ahmed Solyman⁵

ABSTRACT

This study investigates the problematic characteristics of contemporary methods for remote and portable patient monitoring. The consideration is based on recent breakthroughs in information technology and progressive strategies for processing and storing biomedical data. The proposed system represents the Medicine 4.0 concept's next technological leap. Existing methods for remote and portable monitoring of a patient's status have several vital disadvantages in system flexibility and the convenience of processing and evaluating biomedical data, according to an analysis of these systems. The authors have created a new concept for a Remote Patient Monitoring System (RPMS) that allows for undetectable wear during the patient's daily activities. Small modules comprising a microcontroller and a collection of medical sensors transfer data in real-time via wireless Internet of Things (IoT) technologies to a cloud service for the attending physician's processing and visualization convenience. Based on the proposed concept, the authors created a structural diagram of the experimental RPMS and its built prototype. Amazon Web Services (AWS) is used for the real-time processing of biomedical patient data and its subsequent analysis using a graph-based information visualization system. The performed experimental procedure confirmed that the developed experimental RPMS has minimal latency in transmitting data to AWS; it can alert both the patient and the physician about the need for emergency intervention or treatment adjustments, even if critical indicators are detected. Additionally, the proposed system can incorporate components of expert systems and Artificial Intelligence (AI) systems. The authors advocate using the accomplished system for functional diagnostics specialists, paramedics, and cardiologists in medical facilities and the military medical system for rapid diagnosis and direct monitoring of troops' health state on the battlefield.

KEYWORDS

Medicine 4.0; Remote Patient Monitoring (RPM); sensor; microcontrollers; Internet of Things; Amazon Web Services (AWS)

This paper mainly presents the problematic characteristics of contemporary methods for remote and portable patient monitoring and the Medicine 4.0 concept's next technological leap.

The current applied medical systems for remote and portable patient monitoring face serious disadvantages, mainly in the system flexibility, the convenience of processing, and evaluating biomedical data. Consequently, the authors have created a new concept for a Remote Patient Monitoring System (RPMS) that allows for undetectable wear during the patient's daily activities and implemented its prototype for small modules including a microcontroller and a collection of medical sensors transferring data in real-time via wireless Internet of Things (IoT) technologies to a cloud service, where Amazon data were used. Moreover, the performed experimental procedure confirmed that the developed experimental RPMS has minimal latency in transmitting data to Amazon Web Services (AWS) as it can alert both the patient and the physician about the need for emergency intervention or treatment adjustments, even if critical indicators are detected.

1 Introduction

The capacity to combine technology, data, and information flows to Society 5.0 is characterized by modernity. Society 5.0 is a socioeconomic and cultural development strategy centered on employing digital technologies in all sectors of life^[1]. One program within the Society 5.0 framework solves social problems and the problems that arise from them, such as a reduction in fertility, population aging, and the preservation of active longevity for productive participation in society^[2], which is where the concept of Medicine 4.0 has been proposed to apply advanced information, devices, and digital technologies to diagnose and treat patients^[3,4]. Various strategies and approaches can be employed concurrently with this concept^[5-9].

References [10–12] provided a good and detailed background for techniques to improve the application of healthcare field by improving the physical layer technique^[10], data transfer and provenance^[11], and transparent artificial intelligence in ambient intelligence systems^[12].

1 College of Information Technology, Amman Arab University, Amman 11937, Jordan

2 Faculty of Engineering, Zarqa University, Zarqa 13110, Jordan

3 College of Engineering, University of Business and Technology, Jeddah 23847, Saudi Arabia

4 Department of Media Systems and Technology, Kharkiv National University of Radio Electronics, Kharkiv 61166, Ukraine

5 Department of Electrical and Electronics Engineering, Faculty of Engineering and Architecture, Nişantaşı University, İstanbul 34398, Türkiye

Address correspondence to [Hani Attar, Hattar@zu.edu.jo](mailto:Hani.Attar@zu.edu.jo)

© The author(s) 2025. The articles published in this open access journal are distributed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>).

In Refs. [13–15], more advanced technologies were proposed, such as deep learning to identify the most effective genes by the support of soft ensemble with a novel deep neural network^[13], multiservice data traffic evaluation analysis method^[14], and IoT protection and encryption^[15].

In addition, El-Rashidy et al.^[16] did an analytical investigation showing that chronic diseases have been more prevalent because of the recent COVID-19 pandemics. To deal with such a scenario, El-Rashidy et al.^[16] developed an appropriate system based on an RPMS, which increases the efficacy of medical care and the speed of diagnosis and decreases costs. In addition, the work offered in this paper explained the core principles and proposed generalized structures of RPMS, which contain various recommendations that do not permit the development of a functional RPMS system based on the study data.

Taylor et al.^[17] examined 2050 patient cards. They showed that using RPMS systems decreases the number of hospitalizations, length of hospital stays, and emergency department visits by 49% and is effective in Chronic Obstructive Pulmonary Disease (COPD), which reduces the number of hospitalizations compared to other non-invasive monitoring methods^[17]. This study showed the viability and importance of this research field in RPMS using advanced digital technology and modern microcontroller systems.

Zheng and Chang^[18] showed a topology design challenge for RPMS to demonstrate the proposed methodologies' practicability, utility, and compatibility. In addition, sensitivity analysis and management implications are provided using RPMS^[18], where the authors conducted a statistical study using the Multi-Choice Goal Programming (MCGP) method, resulting in the development of piecewise linear utility functions for the implementation and application of RPMS; however, the system's development and design recommendations were not supported. Majeed and Aish^[19] created an RPMS prototype utilizing a Wireless Body Area Network (WBAN). The created RPMS is based on the microcontroller modules Arduino Uno and ESP8266 and has an LCD screen with a resolution of 128 pixel×64 pixel; and the module is worn on the wrist. The obtained data are transferred through a local Wi-Fi network to a computer, where Microsoft Excel is used to generate graphs^[19]. The solution proposed in Ref. [19] is for Medicine 4.0. The concept has various flaws; foremost, it has enormous overall dimensions because of the usage of an LCD screen; here, information is presented on the user's mobile device. The second flaw, from the author's perspective, is lack of cloud services for data storage and analysis. The proposed solutions utilized a tabular approach of storing information in Microsoft Excel, which restricts the application and access to biological information on the patient's real-time status, raising the danger of notification delays to the attending physician. Shah et al.^[20] provided an example of implementing software-based RPMS during COVID-19 pandemics. Shah et al.^[20] identified several results, including a reduction in the number of phone calls from an average of nine to four calls during the monitoring period, resulting in an average savings of 47.60 working hours and no reports of rehospitalization or death. The developed software RPMS functions as a remote chatbot with a physician, despite the absence of sensors, a system for receiving and storing data for further analysis and visualizations. In the developed RPMS, the user enters the data, follows the doctors' directions, and is presented with a database of answers to various typical questions regarding the progression and complications of COVID-19. Based on their analysis, it can be stated that the study applies to the creation of concepts and an experimental model of RPMS within the Medicine 4.0 framework, using sophisticated information technologies for data storage and analysis.

The rest of the paper is organized as follows. Section 2 introduces the development of concepts and structure of the RPMS. Section 3 gives RPMS algorithm generalization development. Section 4 explains the software development for the proposed technique with ESP-32 DevKit 1 Node MCU. Section 5 explains the development of the Human-Machine Interface (HMI) RPMS for analyzing biomedical information based on the Amazon Web Services (AWS) Amazon Quick-Sight. Section 6 illustrates the experimental system and the collected data. And finally, Section 7 concludes the proposed work in this paper.

During the analysis of publications over the past 5 years, the authors noticed that existing RPMS systems do not have flexible approaches to patient monitoring. Moreover, such systems have solutions that are based on outdated technologies. Taken together, this does not allow for improvements in the flexibility and ease of transport of RPMS, as well as their effectiveness in remote patient monitoring. Based on the above, the following task was set and needs to be solved: to develop a new RPMS concept by introducing modern microcontroller hardware into it, using new IoT technologies and integrating elements of cyber information security. This will significantly improve the operating methodology of RPMS, reduce overall dimensions, and also provide data on the patient's condition in real-time, regardless of the location.

2 Development of Concepts and Structure of Remote Patient Monitoring System

Based on the analysis in Section 1, it can be seen that to implement the RPMS within the Medicine 4.0 concepts, it is necessary to implement the following requirements^[21–23]:

- Minimum overall dimensions of the hardware module with the possibility of flexible plugging of the required set of sensors, according to the controlled parameters;
- Flexibility of plugging using wireless data transfer technologies;
- Convenient and understandable Human-Machine Interface (HMI) for monitoring and analysis of data, both for the patient and the doctor;
- Data transfer mode in the “real-time” mode, which should enable to track of the dynamics of the patient's condition remotely;
- Intelligent parameter control mode, with the ability to warn the doctor of “dangerous” or “critical” indicators automatically;
- “Protection” from the deactivation of the sensor while moving;
- Cryptographic protection of transmitted data through unprotected Internet networks.

Considering the specified requirements, the authors propose the following RPMS concept, as shown in Fig. 1.

As depicted in Fig. 1, the patient carries a hardware module with connected sensors that, depending on the user's lifestyle, can be fastened separately to the belt, maintained in free clothes (pockets), or attached to a clip. The information obtained from the sensors is communicated to the module, which subsequently transmits it to the cloud server via the Internet utilizing Wi-Fi or bluetooth wireless data transmission methods. In addition, the patient monitors and analyzes where the indicators are accessible via the patient's mobile phone, and the obtained data are accessible via the hospital server to the attending physician. In this concept, the authors consider the possibility of using the developed RPMS not only at home (hospital) within the local Wi-

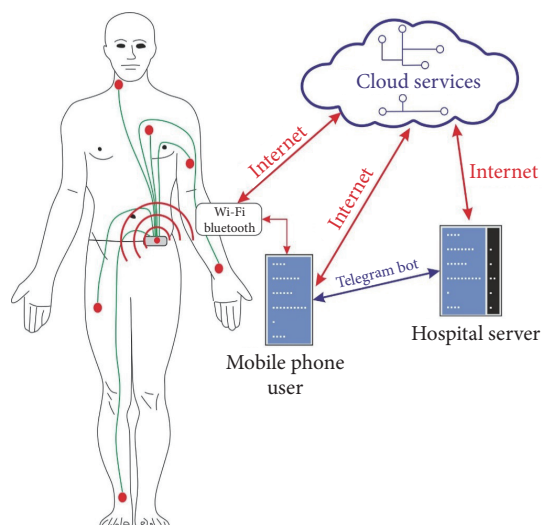


Fig. 1 Main concept of the developed remote patient monitoring system.

Fi network coverage area but also in daily life, when the information received from the sensors is transmitted via bluetooth technology to a user's mobile phone, and then through the operator's 4G network and the Internet to the cloud server. Suppose a consultation or adjustment is required immediately. In this situation, the doctor can communicate with the patient (user) via telegram chat or telegram bot from anywhere globally, enabling timely monitoring of the patient's vital signs. Also, this idea can be employed for remote patient monitoring in the event of pandemics such as SARS-CoV-2 coronavirus (COVID-19), swine flu A/H1N1, and other viral infections that can be transmitted by aerosol droplets^[24,25]. Currently available portable

patient monitors measure various biological data: Electrocardiogram (ECG), heart rate, blood oxygen saturation (SpO₂), Non-Invasive Blood Pressure (NIBP), body temperature (Temp), Heart Rate Contractions (HRC), and Perfusion Index (PI)^[26,27]. Figure 2 depicts the models of standard portable patient monitors for measuring biological data, and Table 1 compares their primary parameters and technical capabilities.

As shown in Table 1, all models of the portable patient monitor for monitoring biological information feature screens that increase the monitor's overall size, and the MD905 and Oxima® 3 have integrated thermal printers for printing data. It is an obsolete technology for storing and transferring data, notably for resolving the problem of monitoring and assessing the patient's state during specific periods, considering current Medicine 4.0 requirements. It is also important to note that just one of the evaluated monitors includes wired and wireless ports for data transfer, whereas bluetooth technology has a limited transfer radius because of its technological properties. All assessed types of portable patient monitors for measuring biological data have a weight ranging between 655 g and 1000 g. This weight makes it difficult to utilize them as a remote monitoring method in the patient's daily life when viewing him in a 24/7 mode. The cost of models ranges from 500 US dollars to 990 US dollars depending on the experimental parameters.

In constructing a structural diagram for the RPMS, the following activities must be completed: Use current techniques for Cloud Services (CS)^[31] and IoT^[32]; And display and store information with aspects of Machine-to-Machine (M2M)^[33] integrations to remove human mistake or inaccuracy in recording and processing information.

As illustrated in Fig. 3, the following structural architecture was

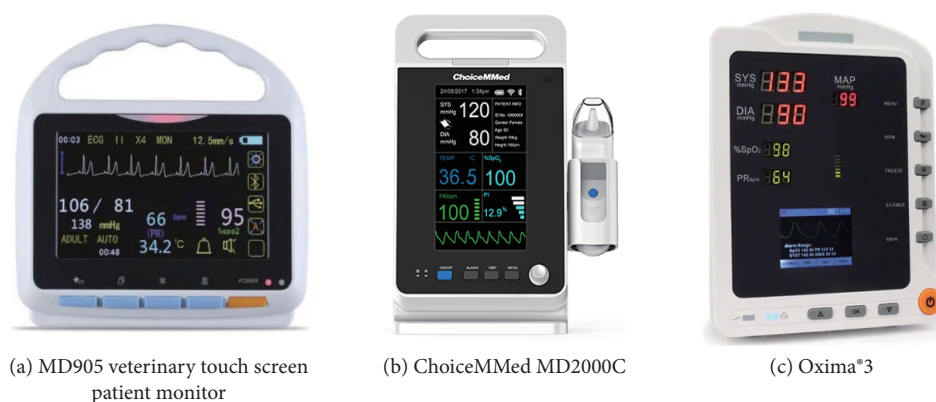


Fig. 2 Portable patient monitors for measuring biomedical information.

Table 1 Characteristics comparison of the existing portable patient monitors for measuring biomedical information.

Parameter	Portable patient monitor model		
	MD905 ^[28]	MD2000C ^[29]	Oxima® 3 ^[30]
Information output	Touchscreen 5.1" color TFT	Touchscreen 7" color TFT	Screen
Measured parameter	ECG, heart rate, SpO ₂ , NIBP, and Temp	PR, PI, NIBP, and Temp	NIBP, SpO ₂ , PR, and Temp
Peripheral information output	Thermal printer	–	Thermal printer
Battery life (h)	15	72	6
Charging time (h)	10	6	8
Overall dimension	209 mm×104 mm×47.5 mm	292 mm×168 mm×148 mm	210 mm×188 mm×107 mm
Weight (g)	655	1000	1000
Price (US dollar)	800–990	600–650	500–590

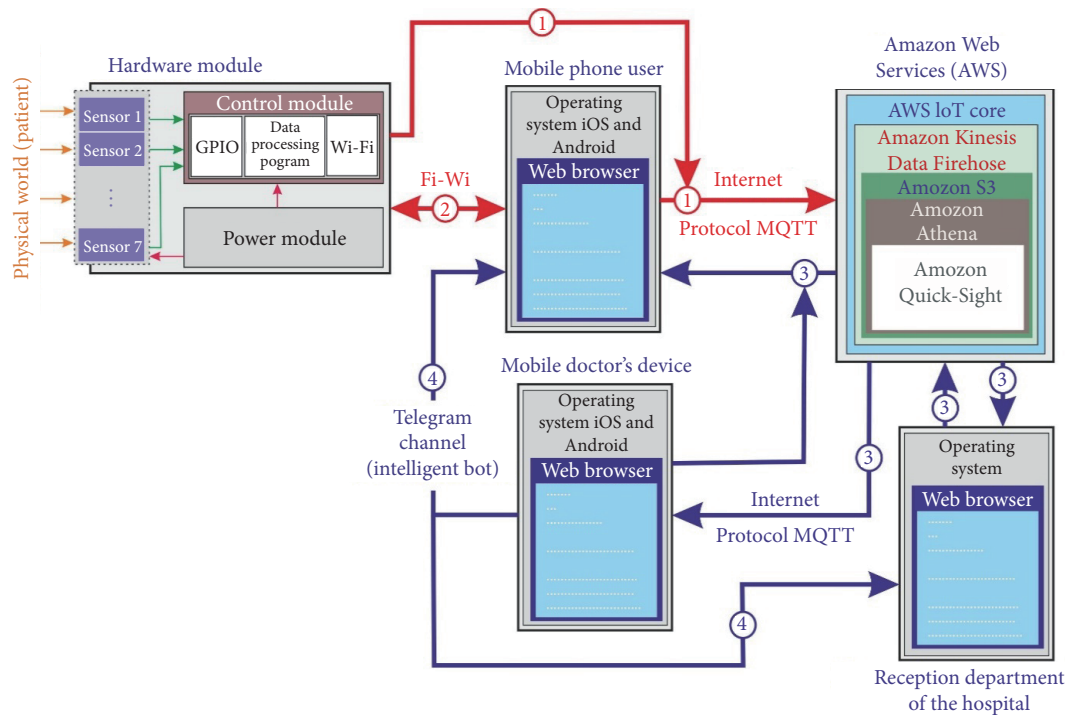


Fig. 3 Structural scheme of remote patient monitoring.

built to implement the suggested concept based on the requirements provided for RPMS for Medicine 4.0. The hardware module, composed of the following modules, is the central component of the developed RPMS. The control module is a microcontroller-based system that handles control operations and data processing. It is equipped with a General-Purpose Input/Output (GPIO) bus used to interface with a sensor array (Sensors 1–7), which monitors and interacts with the real-world environment, specifically the patient^[34]. The data collected from the sensor are processed and analyzed in the data processing program block before being communicated to AWS^[35,36] via the Wi-Fi (bluetooth) module using the Message Queuing Telemetry Transport (MQTT) protocol over the Internet. If direct data transfer from the hardware module is not possible, mobile phone users will transmit data to AWS using a 4G mobile connection if one is available. Thus, the hardware module connects to mobile phone users through Wi-Fi or bluetooth (in which case, both mobile phone users and hardware modules can function as access points). The hardware module will be powered by the power module and will depend on the number of sensors and microcontroller type chosen for this research. It is proposed to use AWS to process, store, and analyze received data. AWS IoT Core^[37] is a service that enables the connection and reading of data from microcontroller devices. The data are then collected in a separate data warehouse, the Amazon Kinesis Data Firehose^[38], and organized with Amazon Simple Storage Service (Amazon S3)^[39]. Data systematization can be changed according to the user's demands, but by default, data are systematized by time stamps, which is highly useful for patient analysis and timeline indicators.

Using Amazon Athena^[40], SQL-based searches to Amazon S3-stored data are constructed, allowing the results of the queries to be displayed as the appropriate structured data prior to Amazon Quick-Sight^[41]. The Amazon Quick-Sight module provides a vast array of visual objects for displaying data, enabling the generation of a data board containing analytical data in graphs and tables to illustrate the evolution of a patient's health over a specified period.

The received data are accessible via the standard Hyper-Text

Transfer Protocol (HTTP)^[42] to both the mobile doctor's device and the hospital's stationary computer to an allowed user (reception department of the hospital). Using the telegram channel, the physician can remotely contact the patient and the hospital in emergency cases (intelligent bot). The choice of using the telegram channel for receiving data from portable patient monitors can be justified by its wide coverage, convenient access to data, and security of transmission. As a popular instant messaging platform, telegram provides the ability to send real-time data directly to multiple users and encryption to protect the privacy of medical data. Here, a client-server model based on a "thin client"^[43] is built, allowing the basic requirements for mobile devices (mobile doctor's device and mobile phone user), the availability of any operating system (OS iOS, OS Windows, and OS Android), and the standard web browser to be met (Safari, Chrome, and Mozilla Firefox).

After RPMS hardware modules and sensors have been built, examining and selecting the hardware modules and sensors for experimental layout implementation are necessary. Based on the requirements above, such as minimum overall dimensions, the capability of flexible sensor plugging, and the availability of wireless information transmission modules, the selection of microcontroller-based control boards is severely constrained. As stated in Fig. 4, the following hardware components were selected for analysis to create the RPMS control module.

Table 2 displays the results of an investigation of the primary characteristics of the hardware modules ESP-32 DevKit 1 Node MCU, ESP-32S ESP-WROOM-32, and Raspberry Pi Pico. The authors picked ESP-32 DevKit 1 Node MCU to implement the RPMS being built based on these results (Fig. 4a). The selection of this option was influenced by the following considerations: support for wireless network interfaces Wi-Fi: 802.11 b/g/n and bluetooth BLE v4.2 BR / EDR, unlike Raspberry Pi Pico (Fig. 4c), for which you must purchase an additional Raspberry Pi expansion board Pico ESP8266 Wi-Fi HAT^[47], which will increase overall dimensions by at least 2–2.5 times and cost and prevent you from connecting sensors. Comparing the technical



Fig. 4 Hardware modules for implementing RPMS management.

Table 2 Comparison of the key characteristics of hardware modules for the developed RPMS.

Parameter	ESP-32 DevKit 1 Node MCU ^[44]	ESP-32S ESP-WROOM-32 ^[45]	Raspberry Pi Pico ^[46]
Microcontroller	ESP32	ESP32	ARM Cortex M0+
Frequency	80 to 240 MHz	80 to 240 MHz	Up to 133 MHz (48 MHz by default)
RAM (KB)	520	520	264
Built-in memory (MB)	4	4	2
GPIO/GPI (pin)	25/4	25/4	26
Interface	UART, SPI, SDIO, I2C, LED PWM, Motor PWM, I2S, and IR	UART, SPI, I ² C, I ² S, and PWM	SPI, I2C, UART, and PWM
Network interface	Wi-Fi: 802.11 b/g/n Bluetooth: BLE v4.2 BR/EDR.	Wi-Fi: 802.11 b/g/n Bluetooth: BLE v4.2 BR/EDR.	Require optional ESP8266 Wi-Fi HAT expansion module
Voltage supply (V)	5	2.2–3.6	5
Connector	Micro-USB	–	USB 1.1
Dimension	51 mm×28 mm	25.5 mm×18 mm	51 mm×21 mm
Price (US dollar)	8–10	3–5	8–10

specifications of ESP-32S ESP-WROOM-32 (Fig. 4b) and ESP-32 DevKit 1 Node MCU (Fig. 4a), they are comparable; however, for ESP-32S ESP-OM-32 to function, a power supply circuit with an over-voltage protection mechanism must be developed. For rapid programming, the lack of a micro-USB port is crucial. It should be noted that the ESP-32S ESP-WROOM-32 is smaller and less expensive than the selected module, but its use in constructing the experimental prototype would not be helpful.

The subsequent phase analyzes and selects the sensors attached to the patient to monitor his condition. The selection of sensor types and their intended applications will be determined entirely by the need to govern the properties of a particular kind.

In the modern concept of Medicine 4.0, biomedical sensors can be divided into two major groups^[48]:

- Bio-controllable sensors that alter their characteristics directly under the influence of biomedical information coming from the measurement object.
- Energy sensors that actively affect organs and tissues. They create the so-called unmodulated energy flow in the studied organ with strictly defined and time-constant characteristics.

However, any sensor that will be used in the RPMS being developed in the Medicine's context 4.0 concept must satisfy the following metrological requirements^[49]:

- **Sensitivity:** the minor change in the measured parameter that can be consistently detected with this converter;
- **Dynamic range:** the range of input values that are measured without significant distortion from the maximum limit value to the minimum, restricted by the sensitivity threshold or noise level;
- **Accuracy:** the minimum difference between the received values and the output nominal;
- **Reaction time:** the minimum time interval during which the output value is set to the level corresponding to the altered input

value level.

According to the above metrological requirements and the features of the selected ESP-32 DevKit 1 Node MCU in terms of the options for connecting sensors to the GPIO bus to get biomedical information about the patient in real-time, the following test set of sensors was selected, as shown in Fig. 5, and their main technical characteristics are presented in Table 3.

Comparing the specified sensor settings for connection to the ESP-32 DevKit 1 Node MCU reveals that the heart electrical activity (ECG) measuring module AD8232 has an analog output (Fig. 5f). Therefore, the Analog-to-Digital Converter (ADC) ADS1115 is necessary. The ECG data must be transmitted via the I2C interface to the control module^[56].

Pull-up resistors R1 and R2 with a nominal value of 4700 will be employed for the I2C interface in the created circuit; it must be admitted. Also, a brownout is likely if over three sensors must be connected concurrently. Therefore, the authors recommend using a tantalum CMD capacitor with a capacity of 47 μ F and a voltage of 5 V^[57]. Figure 6 depicts the approach for attaching sensors to the ESP-32 DevKit 1 Node MCU based control module of the RPM system under development. For the convenience of visualizations and understanding of the type of signal transmitted to the scheme, it was decided to mark them with the following colors: green—digital signal for data transmission; black—power circuit “ground” or “gnd”; red—power circuit +5 V or “VCC”. The description is added to the text of the article.

The control module is powered via a micro-USB connector using the power supply module (batteries) or a power bank with the following suggested specifications: 5 V, at least 2000 mAh.

Using the BME280 sensor, an experimental prototype of the RPMS control module was created for future study (see Fig. 7). based on this design, a program will be written for processing and transmitting data from the ESP-32 DevKit 1 Node MCU to AWS

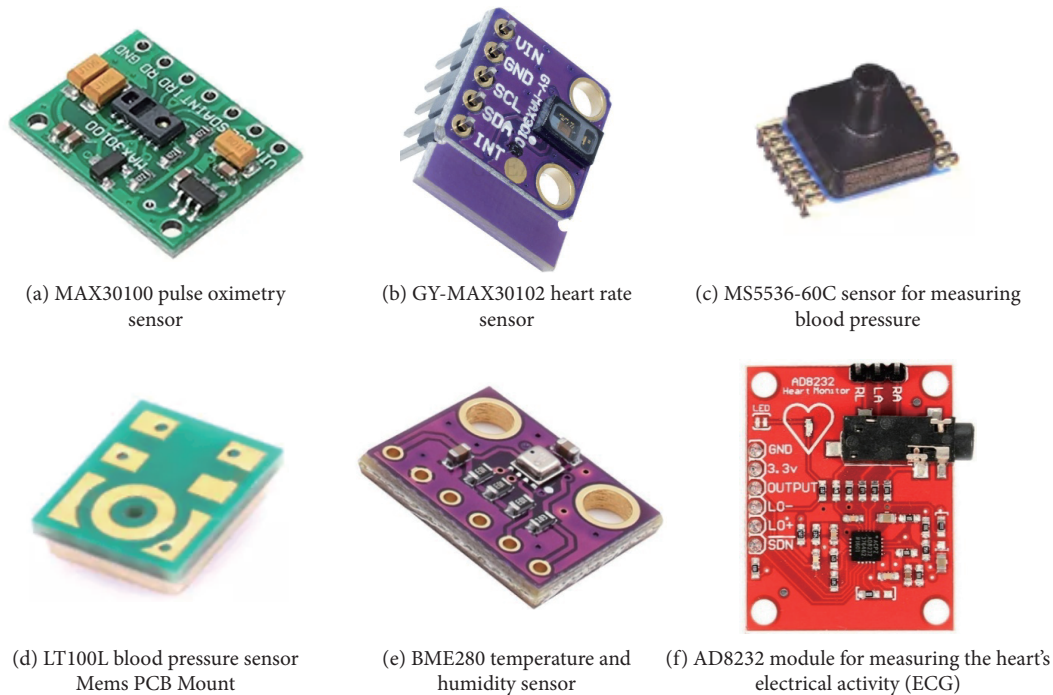


Fig. 5 Sensors for RPMS development.

Table 3 Comparison of the technical characteristics of the selected sensors for the RPMS.

Parameter	Sensor					
	MAX30100 ^[50]	GY-MAX30102 ^[51]	MS5536-60C ^[52]	LT100L ^[53]	BME280 ^[54]	AD8232 ^[55]
Supply voltage (V)	5	3–15	3.3	3–5	3–5	3.3
Current consumption in measurement mode (mA)	1.2	<1	4	<1	2.8	170
Current consumption in standby mode (mA)	10	70	10	10	10	87
Interface	I2C	I2C	SPI	I2C, SPI	I2C, SPI	Analog signal
Overall dimension	18.5 mm×14.4 mm×3 mm	5.6 mm×3.3 mm×1.55 mm	13.4 mm×10.1 mm×10.6 mm	3 mm×5 mm×1.2 mm	15 mm×12 mm×3 mm	3.5 mm×2.8 mm×2.4 mm
Price (US dollar)	3–4	4–5	40–50	4–6	10–15	8–11

for further analysis and presentation of biomedical data.

3 Development of a Generalized RPMS Algorithm

After building the experimental prototype of the RPMS control module for gathering biological information about the patient's condition, the problem of transmission, storage, and later data visualization for analysis by the attending physician must be resolved.

Based on the proposed concept (Fig. 1) and the created RPMS structural scheme (Fig. 3), the authors propose transferring data from the hardware module to ESP-32 via the MQTT protocol using DevKit 1 Node MCU to AWS. MQTT is a lightweight, compact, and open data exchange protocol designed for data transfer to remote sites when minimal code size is required and channel capacity is limited. The benefits mentioned above make it suitable for M2M and Industrial Internet of Things (IIoT) systems. This protocol was selected based on the following characteristics: asynchronous protocol; space-saving messages; stable operation under unreliable data transmission line communication; support for many levels of Quality of Service^[56]; and simple integration with new devices. In addition, the MQTT protocol operates on an application layer over Transmission

Control Protocol/Internet Protocol (TCP/IP) and uses port 1883 by default and port 8883 when connecting via Secure Sockets Layer (SSL)^[59–63]. This involves encrypting data by transforming them into a series of random characters. This set can be decrypted with the AWS server-stored key.

The RPMS control hardware modules are connected to the broadcasting and receiving notification channels created by the AWS IoT core cloud service. Using Amazon Kinesis Firehose, the data received across channels are collected based on predetermined rules. The data are then transmitted to the Extract, Transform, and Load (ETL) service. Amazon Kinesis Data Firehose is an Extraction, Transformation, and Upload (ETU) service that offers dependable collection, transformation, and delivery of streaming data to Data Lake (DL)^[60–64]. Here, it collects unprocessed data, transforms it into time-sorted data, and sends it to the object-oriented Amazon S3 storage. Amazon S3 (Simple Storage Service) facilitates the storage of objects (files) in a cloud provider's emulated file system, ensuring storage stability. Amazon S3 enables machine-loading of data through AWS's internal and user-created external infrastructure, utilizing the access policy and Representational State Transfer Application Programming Interface (REST API), subject to the requisite authorization and access keys.

Time-ordered data are stored in Amazon S3 and analyzed via

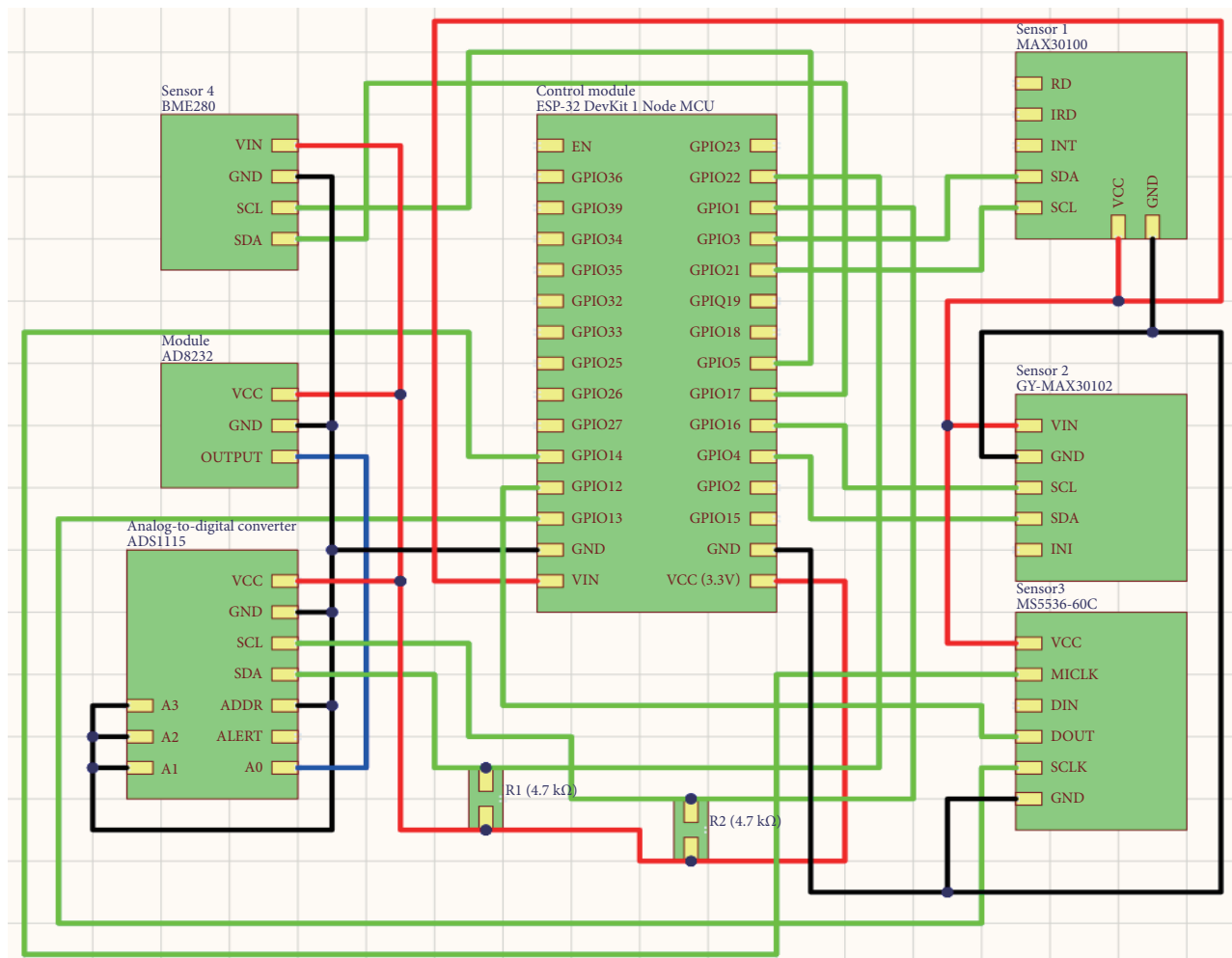
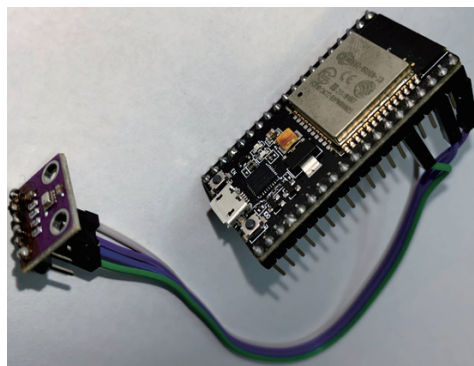
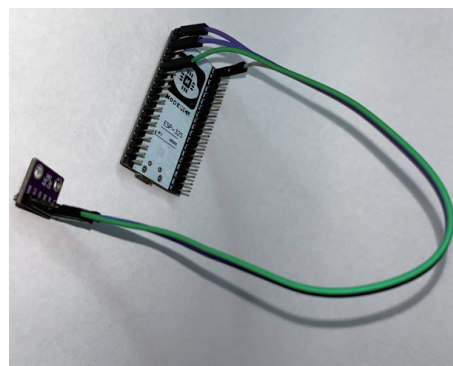


Fig. 6 Scheme of connecting sensors to the RPMS control module.



(a) Top view



(b) Rear view

Fig. 7 RPMS control module experimental prototype with BME280 sensor.

the Amazon Athena query service, which employs the SQL programming language^[61]. It generates tables containing processed data based on Amazon S3 information. Amazon Quick-Sight visualizes data that have been processed with Amazon Athena. AWS's integrated Identity service and Access Management (IAM)^[62] restricts access to patient-specific biomedical information because of its sensitivity. The hospital administrator can assign policies, permissions, and permission sets to particular individuals, groups, and executive roles for services. To access the system, each user has a login and password. Accessing the machine access system requires access and secret keys. The service also provides secure connection endpoints that meet the Medicine 4.0 criterion

for protecting transferred medical data. Figure 8 depicts the developed RPMS generalized operation algorithm.

4 Software Development for ESP-32 DevKit 1 Node MCU

Developing a data collection management program run by ESP-32 DevKit 1 Node MCU, it should be taken into account such features of the developed RPM system as minimal electricity consumption and traffic channel of the Wi-Fi/Bluetooth network. It is also necessary to comply with the principles of secure connection and data storage required for connection. To

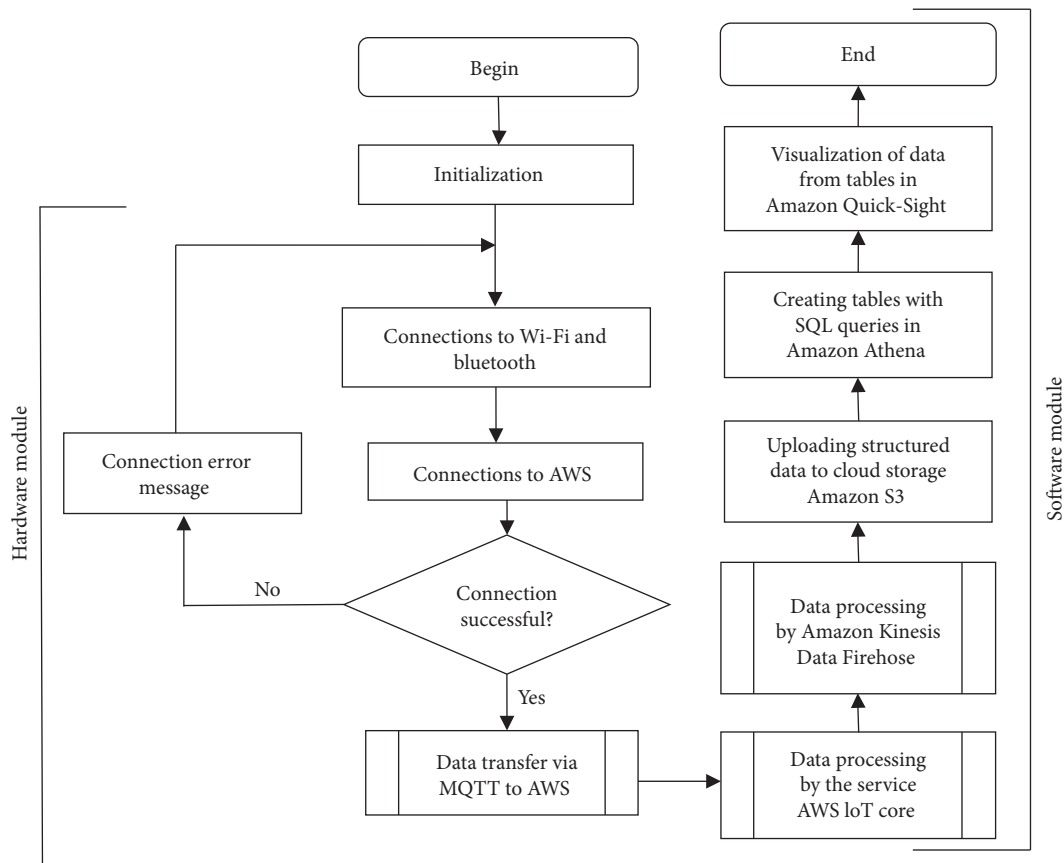


Fig. 8 Generalized operation algorithm of the developed RPMS.

implement this, we will use a Transport Layer Security/Secure Sockets Layer (TLS/SSL) connection by means of the library from Espressif for ESP-32 DevKit called `WifiClientSecure`^[45,47].

The TLS/SSL connection uses the asynchronous keys concept that securely transfer data from the source to the endpoint with a private key verification which will send the hardware to the cloud where Amazon Web Services will verify if the received private key matches the public one containing the access endpoint. In this way, the data transmitted from the hardware will be broadcasted using HTTPS protocol and will be protected against Sniffer and Man-In-The-Middle attacks. The library creates a secure connection that will be used to connect to the AWS IoT Core topic, that in turn receives or publishes data using the `PubSubClient` library, which communicates using the MQTT protocol. The below fragment is a piece of a program code creating a `WifiClientSecure` class object to connect and further use it as a connection to transmit data in the topic.

```

WifiClientSecure net=WifiClientSecure;
PubSubClient client(net);

```

Connection to the AWS root certificate object using the generated RPMS hardware control certificate and a private key will be implemented using the secure connection performed in the following snippet.

```

// Configure WifiClientSecure to use the AWS IoT device
credentials
net.setCACert(AWS_CERT_CA);
net.setCertificate(AWS_CERT_CRT);
net.setPrivateKey(AWS_CERT_PRIVATE);
// Connect to the MQTT broker on the AWS endpoint we
defined earlier
client.setServer(AWS_IOT_ENDPOINT, 8883);
In order to enhance security, it was put in separate secrets.h file

```

the data required to connect to the AWS and contain confidential information, such as data for connecting to the Wi-Fi network, AWS IoT core access point address, AWS root certificate, device certificate, and private key to set up the SSL connection. A fragment of this file is shown in Fig. 9.

Connecting the ESP-32 DevKit 1 Node MCU-based hardware module to the access point is required for relaying biological information about the patient's health state through a Wi-Fi network (local router or mobile phone with the possibility of Wi-Fi sharing). It is planned to employ SSID and password, specified as constants in the ESP-32 DevKit 1 Node MCU firmware, to prevent intruder access to this point. The void `connectAWS()` method presents the connection code fragment below.

```

void connectAWS() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.println("Connecting to Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.print("\n");
}

```

Following the generalized algorithm (Fig. 8), it is necessary to connect to the AWS IoT core service using a tracking code after connecting to the specified Wi-Fi point.

```

client.setCallback(message handler);
Serial.println("Connecting to AWS IOT");
while (!client.connect(THINGNAME)){
  Serial.print(".");
  delay(100);
}
if (!client.connected()) {
  Serial.println("AWS IoT Timeout!");
  return;
}

```

After the successful connection, we implement a function

```

1  #include <pgmspace.h>
2
3  #define SECRET
4  #define THINGNAME "esp32"
5
6  const char WIFI_SSID[] = "*****";
7  const char WIFI_PASSWORD[] = "*****";
8  const char AWS_IOT_ENDPOINT[] = "a2v4p8l24x2d9k-ats.iot.eu-west-2.amazonaws.com";
9
10 // Amazon Root CA 1
11 static const char AWS_CERT_CA[] PROGMEM = R"EOF(
12 -----BEGIN CERTIFICATE-----
13 MIIDQTCCAimgAwIBAgITBmyfz5m/jAo54vB4ikPmljZbyjANBgkqhkiG9w0BAQsF
14 ADA5MQswCQYDVQQGEwJVUzEPMA0GA1UEChMGQWlhem9uMRkwFwYDVQQDExBBBWF6
15 b24gUm9vdCBDQSAxMB4XDTE1MDUyNjAwMDAwMFoXDTE1MDUyNjAwMDAwMFowOTEL
16 MAkGALUEBhMCVVMxMzE1MDUyNjAwMDAwMFoXDTE1MDUyNjAwMDAwMFowOTEL
17 b3QgQ0EgMTCCASIwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBALJ4gHHKXNj
18 ca9HgFB0fW7Y14h29Jl091ghYP10hAEvRAIthtOgQ3pOsQTNroBvo3bSMgHFz2M
19 9O6II8c+6zfltRn4SW1w3te5djdYZ6k/oI2peVKVuRF4fn9tBb6dNqcmzU5L/qw
20 IFAGbHrQgLKm+a/sRxmPUDgH3KKHOVj4utWp+UhnMJbulHheb4mjUcAwhmahRwa6
21 VOujw5H5SNz/0egwLX0tdHA114gk957EWW67c4cX8jJGKLhD+rcdqsq08p8kDill
22 93FcXmn/6pUCyziKrlA4b9v7LWlBxcceVOF34GfID5yHI9Y/QCB/IIDEgEw+OyQm
23 jgSubJrIqg0CAwEAAANCMEEAwDwYDVROTAQH/BAUwAwEB/zAOBgNVHQ8BAf8EBAMC
24 AYYwHQYDVROBByEFIQYzIU07LwMlJQuCFmcx7IQTgoIMAOGCSqGSIb3DQEBCwUA
25 A4IBAQCjY8jdaQZChGsV2USggNiMOruYou6r4lK5IpDB/G/wkYUu0yKGX9rbxenDI
26 U5PMCCjYmCXPI6T53iHTfIUJrU6adTrCC2qJeH2ERxhlb1lBjtt/msv0tadQ1wUs
27 N+gDS63pYaACbvXy8MWy7Vu33PqUXHeeE6V/Uq2V8viTO96LXFvKWLJbYK8U90vv
28 o/ufQJvTMVT8QtPHR8jrdkPSHCa2XV4cdFyQzR1bldZwgJcJmApzyMZFo6IQ6XU
29 5MsI+yMRQ+hDKXJioaldXgjUkK642M4UwtBV8ob2xJNDd2ZhwLnoQdeXeGADbkpy
30 rQXRfboQnoZsG4q5WTP468SQvVG5

```

Fig. 9 A fragment of the secrets.h file that contains confidential information for connecting to AWS.

responsible for converting the received biomedical information about the patient's health state into JSON format. The function of converting data to JSON format is given below.

```
void messageHandler(char* topic, byte* payload, unsigned int
length) {
  Serial.print("incoming: ");
  Serial.println(topic);
  StaticJsonDocument<200> doc;
  deserializeJson(doc, payload);
  const char* message = doc["message"];
  Serial.println(message);
  Sending converted JSON format messages to the AWS IoT
  Core cloud server is implemented in the publishMessage()
  function below.
```

```
void publishMessage() {
  StaticJsonDocument<200> doc;
  doc["Pressure(Pa)"] = p;
  doc["Temperature(*C)"] = t;
  doc["Approx altitude(m)"] = a;
  char jsonBuffer[512];
  serializeJson(doc, jsonBuffer); // print to client
  client.publish(AWS_IOT_PUBLISH_TOPIC, jsonBuffer); }
```

Also, based on the hardware module ESP-32 DevKit 1 Node MCU, the processing of data from sensors (the BME280 sensor in the experimental prototype of the developed RPMS), port speed setting, and IFC protocol address setting are the classic solutions and will not be covered in this article.

5 Development of an HMI RPMS for Analyzing Biomedical Information Based on the AWS Amazon Quick-Sight

Amazon's AWS Amazon Quick-Sight is a customizable business

intelligence solution that enables the development of effective and user-friendly Human-Machine Interfaces (HMIs). The AWS Amazon Quick-Sight designer allows up to 50 datasets per single study and 30 graphical visuals for one desktop and 20 desktops for analysis to facilitate the development and presentation of data to the end user. Based on the AWS Amazon Quick-Sight HMI development paradigm, the following color rendering options are available for RPMS. As a result, let us color the list of fields blue, and the measurement fields display panel green. In the framework of this study, we shall determine that measures are date or text fields that are analogous to finite elements' or indicators' attributes and are utilized to differentiate them. Indicators are numerical numbers used for comparison, measurement, and aggregation. We pick the Athena service as our informational resource. The authors have used the following fields shown in the data panel to facilitate interpreting and visualizing sensor data received from the BME280. Figure 10 depicts an example of a built visualization panel in the AWS Amazon Quick-Sight. Figure 11 depicts the settings for each field displayed in the AWS Amazon Quick-Sight visualization panel.

This enables the display of the relevant data using SQL queries based on the necessary criteria for assessing the dynamics of the patient's health status for the correct diagnosis and prescribing the therapy by the attending physician. It was used by the Amazon Quick-Sight service to display the doctor's and patient's data in conventional graphs; and an implementation example is shown in Fig. 12.

6 Experimental System and Collected Data

Several laboratory studies, ranging from collecting data from the BME280 sensor to its visualization using AWS services, were conducted to validate the accuracy of the judgments made by the built RPMS inside the Medicine 4.0 framework. This laboratory



Fig. 10 Visualization (review) panels in AWS Amazon Quick-Sight.

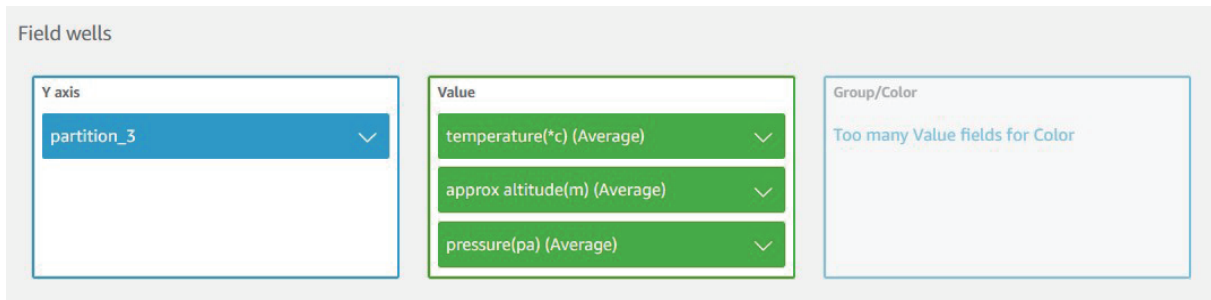


Fig. 11 Detailed setting of each field in the visualization panel.

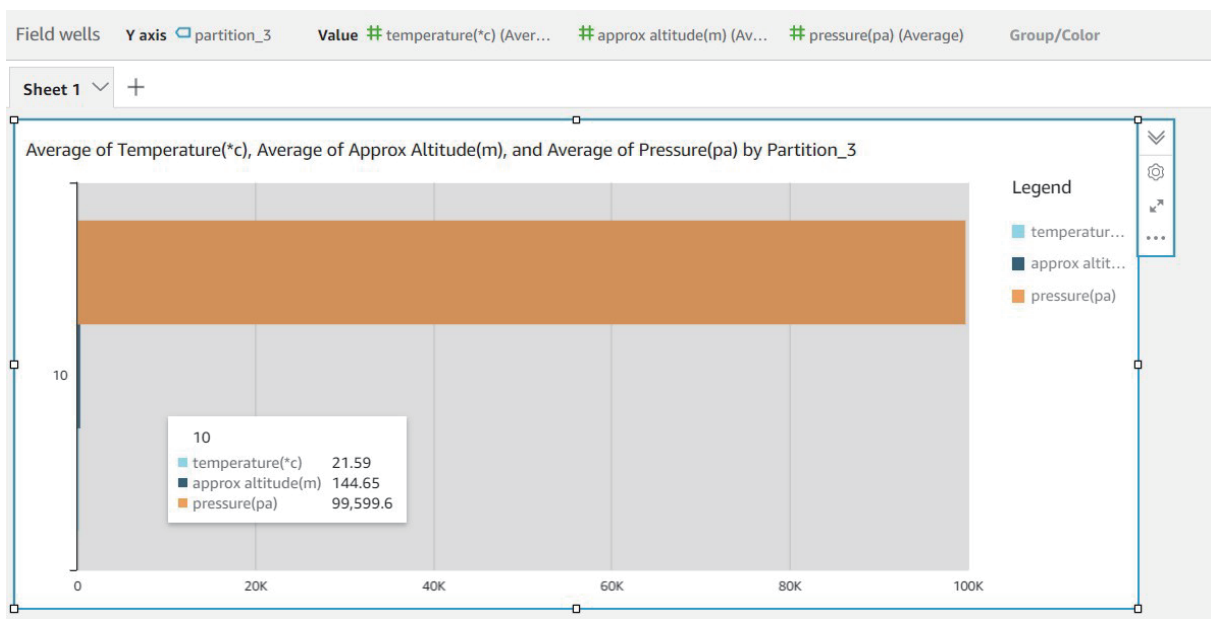


Fig. 12 Developed data visualization window in service designer Amazon Quick-Sight.

experiment ensures the integrity and dependability of legitimate data transfer regarding the patient's physical health status.

In the first stage, we validate that the connection of the ESP-32 DevKit 1 Node MCU to the local Wi-Fi network is correct and that the user may access the AWS IoT core service. In order to accomplish this, it is necessary to connect the ESP-32 DevKit 1 Node MCU modules to a PC and read technical data (in our example, current data from the BME280 sensor) via the monitor serial port (COM) in the Arduino IDE. Figure 13 depicts an application of this phase in practice.

As shown in Fig. 13, the ESP-32 DevKit 1 Node MCU module of the developed RPMS successfully connected with the available Wi-Fi point and AWS IoT core service before broadcasting the current temperature, pressure, and approximate height calculation results from the BME280 sensor.

The last step is to test the functionality of the MQTT protocol and to receive sensor data in the AWS IoT core. Log into the AWS account and launch the MQTT test client, as depicted in Fig. 14.

Since the data are transmitted to the AWS IoT core and redirected according to the rules, it is required to determine if it triggered the AWS Glue Crawler's automatic data collection.

```
COM7
17:12:01.332 -> ...Connecting to AWS IOT
17:12:05.195 -> AWS IoT Connected!
17:12:05.195 -> Temperature = 30.28 *C
17:12:05.195 -> Pressure = 98787.28 Pa
17:12:05.195 -> Approx altitude = 213.46 m
17:12:05.195 ->
17:12:08.184 -> Temperature = 30.27 *C
17:12:08.184 -> Pressure = 98788.87 Pa
17:12:08.184 -> Approx altitude = 213.32 m
17:12:08.229 ->
17:12:11.211 -> Temperature = 30.26 *C
17:12:11.211 -> Pressure = 98790.22 Pa
17:12:11.211 -> Approx altitude = 213.21 m
17:12:11.211 ->
17:12:14.196 -> Temperature = 30.25 *C
17:12:14.243 -> Pressure = 98790.90 Pa
```

Fig. 13 Current data from the BME280 sensor.

Figure 15 depicts data from AWS CloudWatch, which performs an automatic logging function for other services.

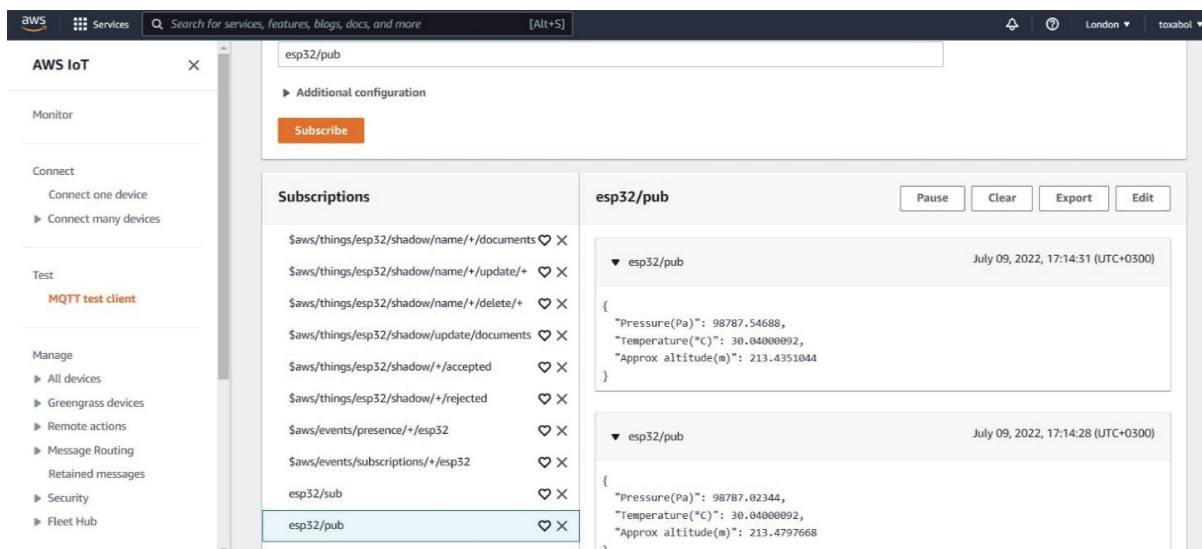


Fig. 14 Results of MOTT test client in AWS IoT core.

▶	Timestamp	Message	
▼	2022-06-14T12:30:48.909+03:00	[c375433f-2682-43f0-af92-cc617ddf9aa4] BENCHMARK : Running Start Crawl for Crawler esp32testcrawler [c375433f-2682-43f0-af92-cc617ddf9aa4] BENCHMARK : Running Start Crawl for Crawler esp32testcrawler	Copy
▼	2022-06-14T12:31:33.211+03:00	[c375433f-2682-43f0-af92-cc617ddf9aa4] BENCHMARK : Classification complete, writing results to database esp32dblonon [c375433f-2682-43f0-af92-cc617ddf9aa4] BENCHMARK : Classification complete, writing results to database esp32dblonon	Copy
▼	2022-06-14T12:31:33.213+03:00	[c375433f-2682-43f0-af92-cc617ddf9aa4] INFO : Crawler configured with SchemaChangePolicy { [c375433f-2682-43f0-af92-cc617ddf9aa4] INFO : Crawler configured with SchemaChangePolicy { "UpdateBehavior": "UPDATE_IN_DATABASE", "DeleteBehavior": "DEPRECATE_IN_DATABASE" }	Copy
▼	2022-06-14T12:31:37.501+03:00	[c375433f-2682-43f0-af92-cc617ddf9aa4] INFO : Table 11 in database esp32dblonon has been updated with new schema [c375433f-2682-43f0-af92-cc617ddf9aa4] INFO : Table 11 in database esp32dblonon has been updated with new schema	Copy
▼	2022-06-14T12:31:37.532+03:00	[c375433f-2682-43f0-af92-cc617ddf9aa4] INFO : Created table 14 in database esp32dblonon [c375433f-2682-43f0-af92-cc617ddf9aa4] INFO : Created table 14 in database esp32dblonon	Copy
▶	2022-06-14T12:31:37.893+03:00	[c375433f-2682-43f0-af92-cc617ddf9aa4] INFO : Partition with values [13] for table 11 in database esp32dblonon has been...	
▶	2022-06-14T12:31:37.975+03:00	[c375433f-2682-43f0-af92-cc617ddf9aa4] INFO : Created partitions with values [[09]] for table 14 in database esp32dblonon...	
▶	2022-06-14T12:31:38.033+03:00	[c375433f-2682-43f0-af92-cc617ddf9aa4] INFO : Created partitions with values [[17]] for table 11 in database esp32dblonon...	
▶	2022-06-14T12:31:39.901+03:00	[c375433f-2682-43f0-af92-cc617ddf9aa4] BENCHMARK : Finished writing to Catalog	
▶	2022-06-14T12:32:47.727+03:00	[c375433f-2682-43f0-af92-cc617ddf9aa4] BENCHMARK : Crawler has finished running and is in state READY	

Fig. 15 Data fragment received by the logging system.

After studying the received logging information, it is workable to infer that the AWS Glue Crawler service does its duties meticulously and stores the received patient health data in the Amazon S3 service. This conclusion can be drawn from the system's second message (Fig. 15).

In addition, the fourth and fifth signals suggest the construction of a new division, which can be interpreted as a sign that the data gathering procedure has been successfully concluded. To accomplish this, let us immediately examine the formed partitions in the Amazon S3 service, as depicted in Fig. 16.

It must also be verified that the data have been uploaded as files to the newly constructed partitions. Figure 17 depicts the fragment containing the added files.

It is also necessary to check the contents of these files to ensure (make sure, make certain, and verify) that the data transfer works properly and the data are saved. Figure 18 shows a content fragment of the file `testdatastreamsp-1-2022-06-11-13-01-17-`

cba0d5dc-0da4-4230-bc82-a535d9bea396.

Since the service keeps the raw data sequentially, they are all recorded in one row sequentially. Based on the file's contents, it can be determined that the AWS Glue Crawler service performed data recording, while the Amazon S3 service saved the actual data. To confirm the liquidity of the retrieved data, we need to visit the Amazon Athena service and query the `esp32dblonion` database, which stores the sorted data for this period and checks its contents. The result is illustrated in Fig. 19.

The outcome, depicted in Fig.19, suggests that Amazon Athena has accomplished its mission of transforming raw data into sorted and ordered data by column and timestamps readily accessible for the attending physician’s analysis in the built RPMS.

The experiment's last phase (stage) is the Amazon Quick-Sight service validation (performance check), during which the tabular data will be displayed in a user-friendly graphical style within the HMI RPMS. Figure 20 shows the final data visualization view in

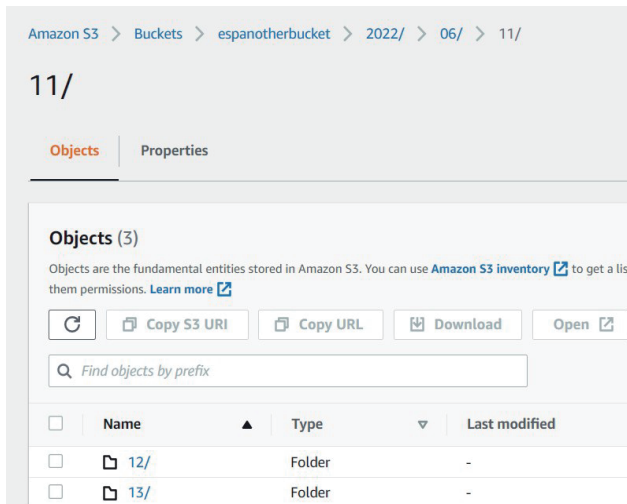


Fig. 16 Partitions created in Amazon S3 while running AWS Glue Crawler.

the Amazon Quick-Sight service.

For clarity, here is the comparison of the RPMS developed in this research to similar solutions, based on the basic parameters that are advanced (required) to such systems within Medicine 4.0 concept. The results of the comparison are presented in Table 4.

As it can be seen from Table 4, the developed RPMS has several advantages over comparable solutions. Comparing the characteristics of the developed experimental RPMS with serial models of portable patient monitors for measuring biomedical information (Fig. 2 and Table 1), you can see that implementing advanced digital and information technologies makes it possible to:

- (1) Simplify the monitoring and data analysis process.
- (2) Reduce overall dimensions and use in patient's everyday life with no inconveniences.
- (3) Access to information will be available in real-time with the

possibility of quickly visualizing the time stamps.

7 Conclusion

In their analysis of published material, the authors determined that building current RPMS within the Medicine 4.0 framework is an urgent task. Solving this problem enables the simplification and automation of obtaining biomedical information about a patient's health status, as well as the remote prevention and treatment of chronic diseases, viral pandemics such as SARS-CoV-2 (COVID-19), and swine flu A/H1N1 because of flexible monitoring of the patient's current indicators and the provision of the medical care. Existing and commercially accessible portable patient monitors for measuring biological data were the subject of the study. The analysis of technical parameters and characteristics revealed several deficiencies, such as the large size, the use of thermal printers for information output, and the limited use of Internet of Things technologies, which led to the assignment of several tasks relating to the development of a new RPMS strategy within the Medicine 4.0 framework. Using cloud services, the Internet of Things, machine-to-machine, and microprocessor systems to gain, process, and analyze biomedical data was the foundation of the new concept. Based on the proposed concept, the remote patient monitoring system structural scheme was developed, the analysis and selection of hardware modules and sensors for the experimental prototype were conducted, a schematic for connecting all elements was designed, and an experimental test prototype utilizing the BME280 sensor was assembled. Amazon web services were selected to store, process, and analyze biomedical data collected over the MQTT protocol and transferred over the Internet. For its implementation, an operation algorithm was created. It selected AWS, based on which all phases will be executed by getting information from the ESP module32 DevKit 1 Node MCU module using AWS IoT core as a raw data array and then visualizing the data as graphs using the Amazon

Name	Type	Last modified	Size	Storage class
testdatastreamsp-1-2022-06-11-13-59-15-5460603e-a65e-44be-914b-c0781aae3aab	-	June 11, 2022, 17:04:17 (UTC+03:00)	7.9 KB	Standard
testdatastreamsp-1-2022-06-11-13-54-14-2251725a-96c6-4fbc-819e-db831f9456dd	-	June 11, 2022, 16:59:17 (UTC+03:00)	8.8 KB	Standard
testdatastreamsp-1-2022-06-11-13-49-13-c085d40a-47d6-409a-9ee4-30104c350c0a	-	June 11, 2022, 16:54:15 (UTC+03:00)	8.8 KB	Standard
testdatastreamsp-1-2022-06-11-13-44-12-0b8ee6e6-cafe-48fd-8154-d7f1e6725029	-	June 11, 2022, 16:49:13 (UTC+03:00)	8.8 KB	Standard
testdatastreamsp-1-2022-06-11-13-39-08-55021de2-09de-4017-8799-cba8daa1dc8b	-	June 11, 2022, 16:44:10 (UTC+03:00)	8.9 KB	Standard
testdatastreamsp-1-2022-06-11-13-34-07-4e61fb5c-1418-43a7-990c-8a9772de24a8	-	June 11, 2022, 16:39:09 (UTC+03:00)	8.8 KB	Standard
testdatastreamsp-1-2022-06-11-13-29-04-6c619528-70aa-4062-a1c6-b1f59d1e3582	-	June 11, 2022, 16:34:06 (UTC+03:00)	6.5 KB	Standard
testdatastreamsp-1-2022-06-11-13-06-18-67a26d9d-94e1-4f57-aaaf-2501bc0e9b8b	-	June 11, 2022, 16:11:19 (UTC+03:00)	5.1 KB	Standard
testdatastreamsp-1-2022-06-11-13-01-17-cba0d5dc-0da4-4230-bc82-a535d9bea396	-	June 11, 2022, 16:06:18 (UTC+03:00)	8.8 KB	Standard

Fig. 17 Data files that were added to the created section.

```

1 [{"Pressure (Pa)": 99457.35938, "Temperature (°C)": 30.73999977, "Approx altitude (m)": 156.6669159, "Pressure (Pa)": 99457.17188,
2 "Temperature (°C)": 30.73999977, "Approx altitude (m)": 156.6832581, "Pressure (Pa)": 99457, "Temperature (°C)": 30.72999954,
3 "Approx altitude (m)": 156.6975403, "Pressure (Pa)": 99456.98438, "Temperature (°C)": 30.72999954, "Approx altitude (m)": 156.695
4 "Pressure (Pa)": 99457.11719, "Temperature (°C)": 30.71999931, "Approx altitude (m)": 156.687851, "Pressure (Pa)": 99457.26563,
5 "Temperature (°C)": 30.70999908, "Approx altitude (m)": 156.6755981, "Pressure (Pa)": 99457.4375, "Temperature (°C)": 30.70999908,
6 "altitude (m)": 156.6602783, "Pressure (Pa)": 99457.1875, "Temperature (°C)": 30.69000053, "Approx altitude (m)": 156.6817169, "P:
7 99456.82813, "Temperature (°C)": 30.69000053, "Approx altitude (m)": 156.7118378, "Pressure (Pa)": 99456.30469, "Temperature (°C)":
8 "Approx altitude (m)": 156.7562561, "Pressure (Pa)": 99456.49219, "Temperature (°C)": 30.70000076, "Approx altitude (m)": 156.740
9 :99456.51563, "Temperature (°C)": 30.70000076, "Approx altitude (m)": 156.7383881, "Pressure (Pa)": 99456.35938, "Temperature (°C)
10 "Approx altitude (m)": 156.7516632, "Pressure (Pa)": 99456.76563, "Temperature (°C)": 30.71999931, "Approx altitude (m)": 156.717

```

Fig. 18 Fragment of the raw data file.

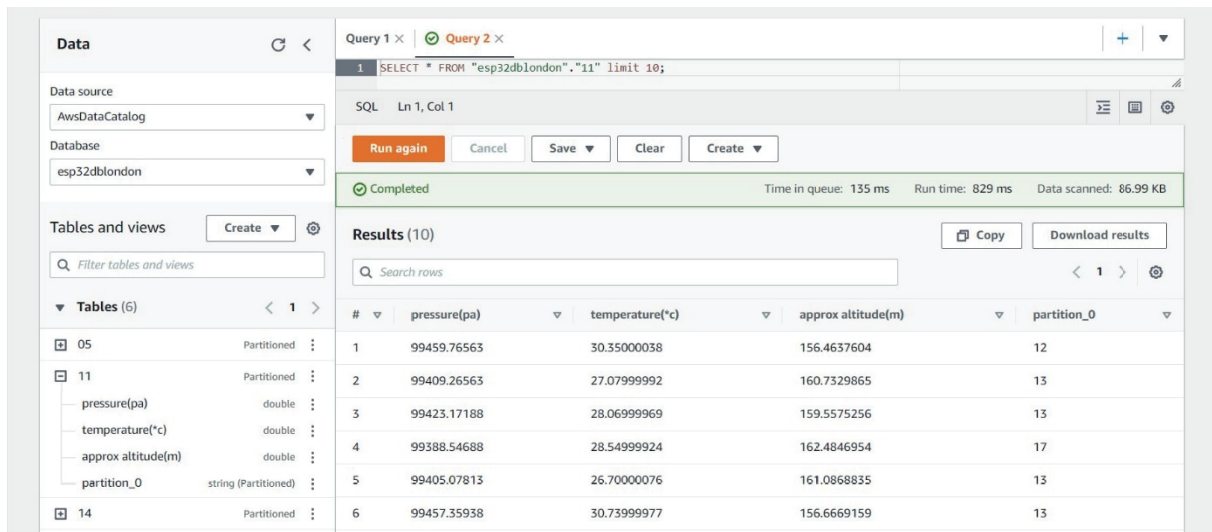


Fig. 19 Executing a select query from the esp32dblonдон database with Amazon Athena.

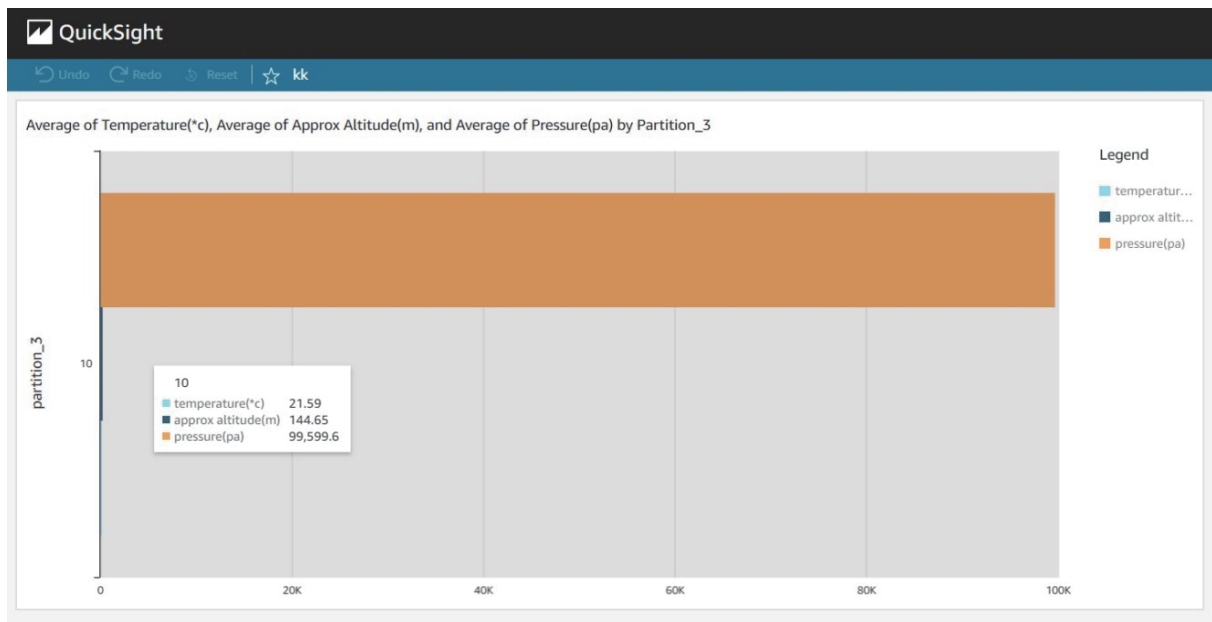


Fig. 20 Test visualization of the patient's biomedical health state in the developed RPMS based on the Amazon Quick-Sight service.

Table 4 Basic parameters of the developed RPMS in comparison with analogs.

Parameter	RPMS		
	Developed RPMS	WVSBN RPMS ^[19]	RPMS ^[65]
Flexible connection capability (option) if required	1 (ESP-32 DevKit 1 Node MCU)	2 (Arduino Uno, ESP8266)	2 (Arduino Uno, ESP8266, GPS)
Information output	6	3	4
Information storage	Yes	No	No
Information retrieval and visualization functions for analysis	Mobile Phone	LSD 128×64	Mobile Phone
Current information remote access	AWS	MS Excel	Web-server on base ESP8266
Data communication protocols	+++	++	+
Weight without power supply (g)	+++	+	++
Hardware module	MQTT	HTTP	HTTP
Plug-in sensor	146–155	242–260	180–200

Note: +++ : maximally convenient option to create search queries, groupings, and visualizations for further analysis, the possibility of flexible access with restrictions of integrity; ++ : a local solution based on the built-in functions of the software product; + : local output of current data by the user's request.

Athena service.

In addition, the sent medical data are secured with asynchronous TLS (SSL) keys. Several experiments were conducted to evaluate the proposed concept's performance. The results showed high-speed information transfer in real-time, a fast and convenient system of data retrieval and sorting by timestamps, and the ability to develop intuitive interfaces for graphical visualization of a patient's health state. In addition, based on research findings, it should use additional sensors to expand the amount of incoming biomedical information about the patient's health and develop an artificial intelligence based self-learning patient diagnosis system.

Thus, the developed RPMS has a number of advantages compared to similar systems, such as flexibility of construction, adaptability to the necessary requirements, and the ability to transmit data in real time 24/7. This makes it possible to obtain more accurate information to track the dynamics of the patient's treatment (illness), with the possibility of remote consultation with a doctor. The authors would also like to emphasize that some functions of the developed RPMS depend on the quality of mobile communications (Internet). This may cause some limitations in functionality, although given the coverage area with access to the Internet, such areas are few at the moment. Nevertheless, this problem encourages further research to improve RPMS and develop its various modifications.

Dates

Received: 1 May 2023; Revised: 15 September 2023; Accepted: 18 September 2023

References

- [1] G. Ioppolo, F. Vazquez, M. G. Hennerici, and E. Andr s, Medicine 4.0: New technologies as tools for a society 5.0, *J. Clin. Med.*, vol. 9, no. 7, p. 2198, 2020.
- [2] Hitachi-UTokyo Laboratory (H-UTokyo Lab.), *Society 5.0: A People-centric Super-smart Society*. Singapore: Springer, 2020.
- [3] The future: medicine 4.0, <https://www.deutschland.de/en/topic/business/medicine-40-in-germany-digitization-and-ai>, 2021.
- [4] C. Juhra, Clinic 4.0: The digital hospital, in *Handbook Industry 4.0*, W. Frenz, ed. Heidelberg, Germany: Springer Berlin, 2022, pp. 843–855.
- [5] H. Attar, A. T. Abu-Jassar, V. Yevsieiev, V. Lyashenko, I. Nevludov, and A. K. Luhach, Zoomorphic mobile robot development for vertical movement based on the geometrical family caterpillar, *Comput. Intell. Neurosci.*, vol. 2022, p. 3046116, 2022.
- [6] V. Lyashenko, O. Kobylin, and Y. Baranchikov, Ideology of image processing in infocommunication systems, in *Proc. Int. Scientific-Practical Conf. Problems of Infocommunications. Science and Technology (PIC S&T)*, Kharkiv, Ukraine, 2018, pp. 47–50.
- [7] S. Khlamov, V. Savanevych, O. Briukhovetskyi, I. Tabakova, and T. Trunova, Data mining of the astronomical images by the CoLiTec software, in *Proc. 6th Int. Conf. Computational Linguistics and Intelligent Systems*, Gliwice, Poland, 2022, pp. 1043–1055.
- [8] V. Akhmetov, S. Khlamov, V. Khramtsov, and A. Dmytrenko, Astrometric reduction of the wide-field images, in *Proc. Conf. Computer Science and Information Technologies*, Lviv, Ukraine, 2020, pp. 896–909.
- [9] A. T. Abu-Jassar, Y. M. Al-Sharo, V. Lyashenko, and S. Sotnik, Some features of classifiers implementation for object recognition in specialized computer systems, *TEM J.*, vol. 10, no. 4, pp. 1645–1654, 2021.
- [10] H. Attar, Efficient physical layer techniques for healthcare applications: Co-operative network coding algorithms and modified equalizers, in *Intelligent Healthcare*, C. Chakraborty and M. R. Khosravi, eds. Singapore: Springer, 2022, pp. 423–454.
- [11] A. L. Michala, H. Attar, and I. Vourganas, Secure data transfer and provenance for distributed healthcare, in *Intelligent Healthcare*, C. Chakraborty and M. R. Khosravi, eds. Singapore: Springer, 2022, pp. 241–260.
- [12] I. Vourganas, H. Attar, and A. L. Michala, Accountable, responsible, transparent artificial intelligence in ambient intelligence systems for healthcare, in *Intelligent Healthcare*, C. Chakraborty and M. R. Khosravi, eds. Singapore: Springer, 2022, pp. 87–111.
- [13] K. Rezaee, G. Jeon, M. R. Khosravi, H. H. Attar, and A. Sabzevari, Deep learning-based microarray cancer classification and ensemble gene selection approach, *IET Syst. Biol.*, vol. 16, nos. 3&4, pp. 120–131, 2022.
- [14] H. Attar, M. R. Khosravi, S. S. Igorovich, K. N. Georgiev, and M. Alhihi, E-health communication system with multiservice data traffic evaluation based on a G/G/1 analysis method, *Curr. Signal Transduct. Ther.*, vol. 16, no. 2, pp. 115–121, 2021.
- [15] G. Samara, R. Quaddoura, M. I. Al-Shalout, K. AL-Qawasmi, and G. A. Besani, Internet of things protection and encryption: A survey, in *Proc. 22nd Int. Arab Conf. Information Technology (ACIT)*, Muscat, Oman, 2021, pp. 1–7.
- [16] N. El-Rashidy, S. El-Sappagh, S. M. R. Islam, H. M. El-Bakry, and S. Abdelrazek, Mobile health in remote patient monitoring for chronic diseases: Principles, trends, and challenges, *Diagnostics*, vol. 11, no. 4, p. 607, 2021.
- [17] M. L. Taylor, E. E. Thomas, C. L. Snoswell, A. C. Smith, and L. J. Caffery, Does remote patient monitoring reduce acute care use? A systematic review, *BMJ Open*, vol. 11, no. 3, p. e040232, 2021.
- [18] X. X. Zheng and C. T. Chang, Topology design of remote patient monitoring system concerning qualitative and quantitative issues, *Omega*, vol. 98, p. 102137, 2021.
- [19] J. H. Majeed and Q. Aish, A remote patient monitoring based on WBAN implementation with internet of thing and cloud server, *Bull. Electr. Eng. Inform.*, vol. 10, no. 3, pp. 1640–1647, 2021.
- [20] S. S. Shah, A. Gvozdanovic, M. Knight, and J. Gagnon, Mobile app-based remote patient monitoring in acute medical conditions: Prospective feasibility study exploring digital health solutions on clinical workload during the COVID crisis, *JMIR Form. Res.*, vol. 5, no. 1, p. e23190, 2021.
- [21] B. McDevitt, L. Moore, N. Akhtar, J. Connolly, R. Doherty, and W. Scott, Validity of a novel research-grade physical activity and sleep monitor for continuous remote patient monitoring, *Sensors*, vol. 21, no. 6, p. 2034, 2021.
- [22] D. R. Seshadri, E. V. Davies, E. R. Harlow, J. J. Hsu, S. C. Knighton, T. A. Walker, J. E. Voos, and C. K. Drummond, Wearable sensors for COVID-19: A call to action to harness our digital infrastructure for remote patient monitoring and virtual assessments, *Front. Digit. Health*, vol. 2, p. 8, 2020.
- [23] L. P. Malasinghe, N. Ramzan, and K. Dahal, Remote patient monitoring: A comprehensive study, *J. Ambient Intell. Hum. Comput.*, vol. 10, no. 1, pp. 57–76, 2019.
- [24] F. Hameed, E. Palatulana, A. Jaywant, R. Said, C. Lau, V. Sood, A. Layton, and A. Gellhorn, Outcomes of a COVID-19 recovery program for patients hospitalized with SARS-CoV-2 infection in New York City: A prospective cohort study, *PM&R*, vol. 13, no. 6, pp. 609–617, 2021.
- [25] W. Ullah, A. Yahya, R. Samikannu, and T. Tlale, Robust and secured telehealth system for COVID-19 patients, in *Data Science for COVID-19*, U. Kose, D. Gupta, V. H. C. D. Albuquerque, and A. Khanna, eds. London, UK: Academic Press, 2021, pp. 337–349.
- [26] Ambiq editorial team, How IoT is improving remote patient monitoring and healthcare outcomes, <https://ambiq.com/how-iot-is-improving-remote-patient-monitoring-and-healthcare-outcomes/>,

- 2022.
- [27] S. Ahammed, N. Hassan, S. H. Cheragee, and A. Z. M. T. Islam, An IoT-based real-time remote health monitoring system, *Int. J. Recent Eng. Sci.*, vol. 8, no. 3, pp. 23–29, 2021.
 - [28] MD905 veterinary touch screen patient monitor, 5.0 inch color TFT screen, https://www.orthopedicdrills.com/product/md905-veterinary-touch-screen-patient-monitor-5-0-inch-color-tft-screen/?gclid=Cj0KCQjwIK-WBhDjARIsAO2sErRjRYbo7lflfz-KrlnaqrOGgXR5jRpSSJqZ_BQLowGt-pstF_tTiUuaAulvEALw_wcB, 2024.
 - [29] Vital signs monitor MD2000C, http://www.choicemed.eu/product_center/256, 2024.
 - [30] Oxima® 3 light weight vital signs patient monitor, <https://www.orthopedicdrills.com/product/oxima-3-light-weight-vital-signs-patient-monitor/>, 2024.
 - [31] A. M. Alkalbani and W. Hussain, Cloud service discovery method: A framework for automatic derivation of cloud marketplace and cloud intelligence to assist consumers in finding cloud services, *Int. J. Commun. Syst.*, vol. 34, no. 8, p. e4780, 2021.
 - [32] P. Ratta, A. Kaur, S. Sharma, M. Shabaz, and G. Dhiman, Application of blockchain and internet of things in healthcare and medical sector: Applications, challenges, and future perspectives, *J. Food Qual.*, vol. 2021, p. 7608296, 2021.
 - [33] V. Mohindru, S. Vashishth, and D. Bathija, Internet of things (IoT) for healthcare systems: A comprehensive survey, in *Recent Innovations in Computing*, P. K. Singh, Y. Singh, M. H. Kolekar, A. K. Kar, and P. J. S. Gonçalves, eds. Singapore: Springer, 2022, pp. 213–229.
 - [34] C. Ünsalan, H. D. Gürhan, and M. E. Yücel, Digital input and output, in *Embedded System Design with ARM Cortex-M Microcontrollers*, C. Ünsalan, H. D. Gürhan, and M. E. Yücel, eds. Cham, Switzerland: Springer, 2021, pp. 53–80.
 - [35] F. H. Yusoff, S. N. K. Kamarudin, and N. Harun, Big data-based image handling—A review of implementation using amazon web services, in *Multimedia Technologies in the Internet of Things Environment, Volume 3*, R. Kumar, R. Sharma, and P. K. Pattnaik, eds. Singapore: Springer, 2022, pp. 95–106.
 - [36] Z. Y. M. Yusoff, M. K. Ishak, L. A. B. Rahim, and O. Ali, Elliptic curve cryptography based security on MQTT system for smart home application, in *Proc. 2022 19th Int. Conf. Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON)*, Prachuap Khiri Khan, Thailand, 2022, pp. 1–4.
 - [37] A. Chakrabarti, P. K. Sadhu, and P. Pal, AWS IoT core and Amazon DeepAR based predictive real-time monitoring framework for industrial induction heating systems, *Microsyst. Technol.*, vol. 29, no. 4, pp. 441–456, 2023.
 - [38] J. J. Paul, Serverless data platforms, in *Distributed Serverless Architectures on AWS*, J. J. Paul, ed. Berkeley, CA, USA: Apress, 2023, pp. 75–93.
 - [39] E. Weisbart and B. A. Cimini, Distributed-something: Scripts to leverage AWS storage and computing for distributed workflows at scale, *Nat. Meth.*, vol. 20, pp. 1120–1121, 2023.
 - [40] N. Bruno, J. Debrodt, C. Song, and W. Zheng, Computation reuse via fusion in Amazon Athena, in *Proc. 2022 IEEE 38th Int. Conf. Data Engineering (ICDE)*, Kuala Lumpur, Malaysia, 2022, pp. 1610–1620.
 - [41] A. Gupta, N. Dhanda, and K. K. Gupta, Ingest and visualize CSV Files using AWS platform for transition from unstructured to structured data, in *Proc. 11th Int. Conf. Emerging Trends in Engineering & Technology - Signal and Information Processing (ICETET - SIP)*, Nagpur, India, 2023, pp. 1–6.
 - [42] A. Tedyyana, O. Ghazali, and O. W. Purbo, A real-time hypertext transfer protocol intrusion detection system on web server, *TELKOMNIKA Telecommun. Comput. Electron. Contr.*, vol. 21, no. 3, pp. 566–573, 2023.
 - [43] E. Bykova, M. Skachkova, I. Raguzin, I. Dyachkova, and M. Boltov, Automation of negative infrastructural externalities assessment methods to determine the cost of land resources based on the development of a “thin client” model, *Sustainability*, vol. 14, no. 15, p. 9383, 2022.
 - [44] Mamta, A. Paul, and R. Tiwari, Smart home automation system based on IoT using chip microcontroller, in *Proc. 2022 9th Int. Conf. Computing for Sustainable Global Development (INDIACom)*, New Delhi, India, 2022, pp. 564–568.
 - [45] E. A. Rodriguez, L. F. M. Fernandez, G. A. C. Orta, A. M. R. Horta, C. B. Barco, K. J. Barrionuevo, D. Cama-Pinto, F. M. Arrabal-Campos, J. A. Martinez-Lao, and A. Cama-Pinto, A platform for inpatient safety management based on IoT technology, *Inventions*, vol. 7, no. 4, p. 116, 2022.
 - [46] C. Bell, Introducing the Raspberry Pi Pico, in *Beginning MicroPython with the Raspberry Pi Pico*, C. Bell, ed. Berkeley, CA, USA: Apress, 2022, pp. 1–42.
 - [47] A. N. M. Hat, W. M. W. Syahidah, C. W. Y. C. W. Zuhelmi, F. Atan, and S. Khadijah, IoT base on air and sound pollution monitoring system, *J. Phys.: Conf. Ser.*, vol. 2319, no. 1, p. 012013, 2022.
 - [48] A. Haleem, M. Javaid, R. P. Singh, and R. Suman, Medical 4.0 technologies for healthcare: Features, capabilities, and applications, *Internet Things Cyber Phys. Syst.*, vol. 2, pp. 12–30, 2022.
 - [49] M. Chandra, K. Kumar, P. Thakur, S. Chattopadhyaya, F. Alam, and S. Kumar, Digital technologies, healthcare and COVID-19: Insights from developing and emerging nations, *Health Technol.*, vol. 12, no. 2, pp. 547–568, 2022.
 - [50] K. V. S. S. Ganesh, S. P. S. Jeyanth, and A. R. Bevi, IOT based portable heart rate and SpO2 pulse oximeter, *HardwareX*, vol. 11, p. e00309, 2022.
 - [51] S. E. Dobbartin-Sanchez and H. A. Chinchay-Espino, Low-cost IoT-based oximeter controlled by NodeMCU and MAX30102 targeting Peru's non-specialized population in 2022, doi: 10.2139/ssrn.4361276.
 - [52] A. S. M. A. Islam, A. M. Afsar, and J. S. Go, Theoretical and experimental investigation of response time in pressure measurement with μ -tubes, in *Proc. Int. Conf. Mechanical Engineering 2011 (ICME2011)*, Dhaka, Bangladesh, 2011, pp. 1–7.
 - [53] A. Lois, E. Sokos, N. Martakis, P. Paraskevopoulos, and G. A. Tselentis, A new automatic S-onset detection technique: Application in local earthquake data, *GEOPHYSICS*, vol. 78, no. 1, pp. KS1–KS11, 2013.
 - [54] M. Tagle, F. Rojas, F. Reyes, Y. Vásquez, F. Hallgren, J. Lindén, D. Kolev, Å. K. Watne, and P. Oyola, Field performance of a low-cost sensor in the monitoring of particulate matter in Santiago, Chile, *Environ. Monit. Assess.*, vol. 192, no. 3, p. 171, 2020.
 - [55] J. Mendes Junior, D. Campos, L. Biassio, P. Passos, P. Júnior, A. Lazzaretti, and E. Krueger, AD8232 to biopotentials sensors: Open source project and benchmark, *Electronics*, vol. 12, no. 4, p. 833, 2023.
 - [56] A. Agnihotri, Human body respiration measurement using digital temperature sensor with I2C interface, *Int. J. Electronics Communication and Computer Engineering*, vol. 4, no. 1, pp. 232–238, 2013.
 - [57] L. Li, Y. Wang, X. Chen, and X. Ren, Research on improvement of configurable I2C controller IP core, in *Proc. IEEE Int. Conf. Control, Electronics and Computer Technology (ICCECT)*, Jilin, China, 2023, pp. 484–489.
 - [58] J. Cardoso, A. Sheth, J. Miller, J. Arnold, and K. Kochut, Quality of service for workflows and web service processes, *J. Web Semant.*, vol. 1, no. 3, pp. 281–308, 2004.
 - [59] R. Dastres and M. Soori, Secure socket layer (SSL) in the network and web security, *Int. J. Computer and Information Sciences*, vol. 14, no. 10, pp. 330–333, 2020.

- [60] V. Khedekar and Y. Tian, Multi-tenant big data analytics on AWS cloud platform, in *Proc. 10th Annual Computing and Communication Workshop and Conf. (CCWC)*, Las Vegas, NV, USA, 2020, pp. 647–653.
- [61] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff, How Amazon web services uses formal methods, *Commun. ACM*, vol. 58, no. 4, pp. 66–73, 2015.
- [62] A. Sankaran, P. Datta, and A. Bates, Workflow integration alleviates identity and access management in serverless computing, in *Proc. Annual Computer Security Applications Conf.*, Austin, TX, USA, 2020, pp. 496–509.
- [63] M. L. Das and N. Samdaria, On the security of SSL/TLS-enabled applications, *Appl. Comput. Inform.*, vol. 10, nos. 1&2, pp. 68–81, 2014.
- [64] H. Mohapatra, Handling of man-in-the-middle attack in WSN through intrusion detection system, *Int. J. Emerg. Trends Eng. Res.*, vol. 8, no. 5, pp. 1503–1510, 2020.
- [65] P. K. Choudhary and J. Vasavada, IoT based smart remote patient monitoring system, *Int. J. Innovations in Engineering Research and Technology*, vol. 8, no. 6, pp. 28–33, 2021.